NEXRAG: A HYBRID RAG CHAT-BOT FOR EDUCATION SYSTEM

PROJECT REPORT

submitted by

ADNAN SAIF
MEK22CS003

AJAS N
MEK22CS006

AJMAL AHAMMED KABEER
MEK22CS007

AL AMEEN
MEK22CS009

to

the APJ Abdul Kalam Technological University
in partial fulfillment of the requirements for the award of Bachelor of Technology
in Computer Science and Engineering

Department of Computer Science and Engineering

MES Institute of Technology and Management, Chathannoor, Kollam

April 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MES INSTITUTE OF TECHNOLOGY AND MANAGEMENT, KOLLAM

CERTIFICATE

Certified that this report entitled *"NEXRAG: A HYBRID RAG CHAT-BOT FOR EDUCATION SYSTEM"* is the report of project presented by **ADNAN SAIF (MEK22CS003), AJAS N (MEK22CS006), AJMAL AHAMMED KABEER (MEK22CS007), AL AMEEN (MEK22CS009)** during the year **2025–2026** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering of the *APJ Abdul Kalam Technological University*.

Muneera
Assistant Professor
Dept. of Computer Science and Engineering
MES Institute of Technology and Management, Kollam

Prof. Ajeena T
Assistant Professor
Dept. of Computer Science and Engineering
MES Institute of Technology and Management, Kollam

Dr. Vineetha G R
Head of the Department
Dept. of Computer Science and Engineering
MES Institute of Technology and Management, Kollam

DECLARATION

We, Adnan Saif, Ajas N, Ajmal Ahammed Kabeer, and Al Ameen, hereby declare that this project report entitled "NEXRAG: A HYBRID RAG CHAT-BOT FOR EDUCATION SYSTEM" is the bonafide work carried out by us under the supervision of Muneera, Assistant Professor, Department of Computer Science and Engineering. We declare that, to the best of our knowledge, the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion to any other candidate. The content of this report is not being presented by any other student to this or any other University for the award of a degree.

Signature:
Name of the Student: Adnan Saif
University Register No: MEK22CS003 of year 2026

Signature:
Name of the Student: Ajas N
University Register No: MEK22CS006 of year 2026

Signature:
Name of the Student: Ajmal Ahammed Kabeer
University Register No: MEK22CS007 of year 2026

Signature:
Name of the Student: Al Ameen
University Register No: MEK22CS009 of year 2026

Signature(s):
Name of Guide(s): Muneera

Countersigned with Dr. Vineetha G R:
Head, Department of Computer Science Engineering
MES Institute of Technology and Management, Kollam Date: _____

ACKNOWLEDGEMENTS

We take this opportunity to express our deep sense of gratitude and sincere thanks to all who helped us to complete the project successfully.

We are deeply indebted to our guide Muneera, Assistant Professor, Department of Computer Science and Engineering, MES Institute of Technology and Management, for excellent guidance, positive criticism, and valuable comments. We are greatly thankful to Dr. Vineetha G R, Head of the Computer Science and Engineering Department, for her support and cooperation.

Finally, we thank our parents, friends, and all those who directly and indirectly contributed to the successful completion of our project.

ADNAN SAIF (MEK22CS003)
AJAS N (MEK22CS006)
AJMAL AHAMMED KABEER (MEK22CS007)
AL AMEEN (MEK22CS009)

Place: Kollam

Date:

ABSTRACT

NexRag is a hybrid Retrieval-Augmented Generation chatbot developed for educational institutions to provide accurate answers to academic and administrative queries from both structured and unstructured knowledge sources. The system combines a knowledge graph for entity-centric institutional facts with a document retrieval pipeline for guidelines, regulations, syllabi, and circulars. User queries are processed through an intent router that decides whether graph querying, document retrieval, or a combined evidence path should be prioritized. Retrieved graph facts and ranked document chunks are then supplied to a locally deployed large language model through Ollama so that the final response remains grounded in institutional evidence rather than unsupported model memory. The implementation uses FastAPI for backend orchestration, React and Vite for the frontend, FAISS and BM25 for hybrid retrieval, a cross-encoder for reranking, and Apache Jena Fuseki for SPARQL-based graph access. The system also exposes route labels, source transparency, confidence indicators, and graph-editing support to improve trust and maintainability. Experimental verification from the repository confirms indexed academic documents, working query routing, live graph-backed answers, and effective retrieval for handbook and seminar-related questions. The project demonstrates that a local-first hybrid academic assistant can improve information access, reduce hallucination, and provide a scalable foundation for institution-specific question answering.

Key words: Retrieval-Augmented Generation, Knowledge Graph, BM25, Large Language Model, Academic Chatbot.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Sl. No	Abbreviation	Full Form
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	BM25	Best Matching 25
4	B.Tech	Bachelor of Technology
5	DFD	Data Flow Diagram
6	FAISS	Facebook AI Similarity Search
7	KG	Knowledge Graph
8	LLM	Large Language Model
9	PDF	Portable Document Format
10	RAG	Retrieval-Augmented Generation
11	RDF	Resource Description Framework
12	SPARQL	SPARQL Protocol and RDF Query Language
13	TTL	Terse RDF Triple Language
14	UI	User Interface

CHAPTER 1: INTRODUCTION

1.1 OVERVIEW

Large language models have changed the way users expect to access institutional information. Instead of manually browsing notices, handbooks, circulars, and course files, users increasingly prefer to ask direct questions and receive concise, evidence-backed answers. This expectation is especially visible in academic institutions, where students and faculty repeatedly seek operational information such as attendance rules, seminar procedures, course ownership, report format, and departmental responsibilities. However, institutional knowledge is rarely stored in a single uniform form. Some facts are relational and structured, while other information exists only within lengthy PDF documents. The project domain of NexRag therefore lies at the intersection of academic information systems, information retrieval, knowledge representation, and local conversational artificial intelligence.

Retrieval-Augmented Generation (RAG) has emerged as a practical response to the limitations of standalone language models in knowledge-intensive tasks. Instead of relying entirely on parametric memory, a RAG system retrieves external evidence and uses it to ground the final response, thereby improving factuality and updateability (Lewis et al., 2020). In academic environments this is particularly important because institutional policies, syllabi, guidelines, and staff responsibilities change over time. A purely generative system may produce fluent but inaccurate answers, whereas an evidence-grounded system can align responses with current institutional documents and structured records.

Academic query answering also requires more than document retrieval alone. Questions such as "Who teaches Computer Graphics?" or "Which department offers this course?" depend on explicit entity relationships rather than only descriptive text. Knowledge graphs are well suited to such cases because they represent entities and links in a machine-readable form that supports direct querying and interpretable reasoning (Hogan et al., 2021). NexRag addresses this dual requirement by combining a knowledge graph for structured institutional facts with a retrieval

pipeline for unstructured academic documents, and by using a local language model to synthesize the retrieved evidence into a user-facing response.

Unlike open-domain assistants, a campus-facing assistant operates within a bounded but high-accountability knowledge environment. The indexed corpus may be relatively small, yet the cost of a wrong answer is high because misinformation about attendance, seminar submission, or course responsibility can directly affect academic compliance. The project domain therefore values authority, traceability, and maintainability as strongly as conversational fluency. For this reason, NexRag is framed not as a generic chatbot, but as an evidence-controlled academic information system.

The domain is also relevant from a systems perspective because it requires multiple engineering concerns to be solved together. The assistant must support document ingestion, structured data maintenance, evidence ranking, answer generation, and interface transparency within one workflow. A project in this domain therefore cannot be evaluated only as a chatbot; it must be understood as an integrated academic information system whose success depends on retrieval quality, fact representation, operational control, and user trust.

1.2 BACKGROUND

Traditional academic information access follows a document-first model. Users search handbook PDFs, seminar instructions, course files, and departmental notices, often without knowing which source contains the required answer. This creates delay, repeated faculty intervention, and inconsistent interpretation of rules. Even when information is technically available, discoverability remains weak because official documents use formal language that may not match the way users phrase their questions.

Neural retrieval research has shown that dense embedding methods can bridge this vocabulary mismatch by mapping semantically related queries and passages into a shared vector space (Karpukhin et al., 2020). Sentence-level embedding approaches further made semantic retrieval efficient enough for practical search systems (Reimers and Gurevych, 2019). At the same time, lexical methods such as BM25 remain valuable because institutional rules often contain exact phrases,

codes, and formal terminology that should strongly influence ranking (Robertson and Zaragoza, 2009). Modern grounded assistants therefore benefit from hybrid retrieval rather than commitment to a single search strategy.

Another important development is neural reranking, where a first-stage candidate set is refined using a more accurate query-passage interaction model. BERT-based rerankers have shown strong precision for passage selection and are particularly useful when users need the most authoritative supporting evidence rather than a long list of loosely related results (Nogueira and Cho, 2019). In a university assistant, this matters because a small ranking error may change the meaning of a policy explanation or direct the user to the wrong guideline.

The importance of local-first deployment adds a practical institutional dimension. Cloud-dependent assistants raise concerns about privacy, cost, internet dependence, and control over internal documents. A local architecture that keeps the model, vector index, and knowledge graph within the institutional environment aligns better with educational settings where budget and governance constraints matter. NexRag is therefore important not only as a question-answering system, but also as a demonstration of how grounded AI can be implemented in a controllable and institution-ready form.

The educational importance of such a system is equally substantial. A usable institutional assistant reduces repetitive faculty queries, improves access for new students, and promotes consistent interpretation of regulations. At the project level, it also demonstrates how information retrieval, semantic technologies, prompt engineering, and software deployment can be combined in an application with direct practical value rather than in disconnected experimental modules.

From an information-systems perspective, the importance of the problem also lies in the diversity of its users. First-year students, final-year project teams, faculty members, and coordinators do not formulate queries in the same way, even when they refer to the same regulation or document. A useful system must therefore translate natural, sometimes informal, user phrasing into institutionally meaningful evidence. This requirement strengthens the case for hybrid retrieval, reranking, and explicit source presentation in the proposed system.

1.3 MOTIVATION

The primary motivation for NexRag is the gap between the availability of academic information and its actual usability. Students repeatedly need answers to ordinary but high-impact questions involving attendance requirements, seminar instructions, project-report formatting, syllabus details, and faculty-course mapping. These questions should be answerable immediately, yet in practice they often require searching multiple PDFs or contacting faculty members. This creates avoidable communication overhead and delays access to routine academic guidance.

A second motivation arises from the limitations of generic conversational AI. Although large language models can generate fluent explanations, they are not reliable sources of institution-specific information unless grounded in approved evidence. An academic assistant must therefore do more than converse naturally; it must retrieve relevant evidence, preserve factual consistency, and expose the source of its answer. This requirement directly motivated the hybrid design of NexRag.

The project was also motivated by the need to combine multiple knowledge modalities in one coherent system. Pure document retrieval is weak for explicit relations, while pure graph querying is weak for descriptive procedural content. A practical academic assistant must support both. NexRag was therefore conceived as an evidence-routing system in which query intent determines whether graph search, document retrieval, or a combined path should dominate response generation. This makes the system useful not only as a chatbot, but as a structured academic information service.

An additional motivation was transparency. Many AI systems return polished responses without showing how they were produced. In an academic context, this is unsatisfactory because users often need to verify whether an answer is based on an approved guideline, a handbook rule, or a structured institutional fact. NexRag was therefore designed to surface route labels, source snippets, and system-status signals so that response quality can be assessed rather than merely assumed.

The project is also motivated by feasibility and academic relevance. A final-year engineering project should solve a real problem using methods that are contemporary, technically significant, and practically deployable with limited infrastructure. Hybrid

RAG satisfies these conditions because it combines modern AI techniques with implementable software components such as FastAPI, FAISS, SPARQL, and local model serving. The resulting system demonstrates both current research relevance and practical engineering discipline.

1.4 PROBLEM STATEMENT

Academic institutions maintain critical information in heterogeneous forms such as handbooks, syllabus documents, seminar guidelines, circulars, and departmental records. Users seeking answers to academic questions often face difficulty locating the right source, identifying the relevant passage, and verifying whether the answer is institution-specific and current. Generic chatbots cannot guarantee factual alignment with institutional rules, while ordinary search systems lack conversational interaction and struggle with semantic intent.

The problem addressed in this project is therefore to design a local-first academic assistant that can answer institutional queries accurately, transparently, and efficiently by combining structured knowledge graph reasoning, unstructured document retrieval, and grounded language generation. The system must classify queries, retrieve the appropriate evidence, present clear answers, and remain maintainable as documents and institutional facts evolve.

In operational terms, the problem also includes orchestration. The system must decide when graph evidence is sufficient, when document retrieval should dominate, and when both sources should be combined. It must expose those decisions through a usable interface and support document and graph updates without compromising consistency. This broader formulation is what separates NexRag from a conventional FAQ bot or standalone search bar.

1.5 CHALLENGES

Existing systems face several limitations that justify the proposed work. The first is fragmentation of knowledge sources. Academic information is scattered across documents and records with different structures and vocabularies, making centralized access difficult. The second is unsupported generation: large language models can produce plausible answers even when evidence is weak, which is risky

in a fact-sensitive academic setting (Lewis et al., 2020; Es et al., 2024).

The third challenge is semantic mismatch between user phrasing and institutional language. Queries such as "minimum attendance rule" may correspond to formal handbook wording on eligibility or condonation. Dense retrieval helps address this gap, but lexical specificity remains necessary for codes, formal rule names, and exact policy terms. The fourth challenge is relational reasoning. Queries about courses, faculty, departments, and regulations often require explicit entity links that are handled more naturally by a knowledge graph than by text similarity alone.

The fifth challenge is maintainability. Institutional knowledge changes through new regulations, updated faculty assignments, and revised guidelines. A useful assistant must therefore support document updates, graph maintenance, and index rebuilding without redesigning the whole system. The sixth challenge is evaluation: hybrid RAG systems must be judged not only by fluency, but also by retrieval quality, faithfulness, graph availability, and interface transparency (Es et al., 2024). NexRag addresses these challenges through a modular hybrid architecture rather than a single-model solution.

Another practical challenge is coordination among local services. A local-first assistant depends on the simultaneous health of the backend API, vector artefacts, graph endpoint, and language-model runtime. Even if one component is temporarily unavailable, the system should degrade gracefully and communicate its state. This requirement influenced the design of status endpoints, evidence-based route labels, and operational logging in the proposed system.

Trust therefore emerges as a technical requirement rather than a purely interface-level concern. An academic assistant must answer accurately when evidence exists, but it must also signal uncertainty when evidence is weak, unavailable, or contradictory. Systems that conceal retrieval failure behind confident language are unsuitable for policy-sensitive institutional use. This challenge directly motivates the confidence indicators, source traces, and route visibility adopted in NexRag.

1.6 OBJECTIVES

The main objective of NexRag is to develop a robust academic assistant that answers institutional queries through evidence-backed hybrid retrieval. To achieve this, the project seeks to build a structured knowledge graph for departments, faculty, courses, and regulations; implement a document ingestion and indexing pipeline for academic PDFs; support both semantic and lexical retrieval; and use a local language model to generate grounded answers from the retrieved context.

The project also aims to classify user intent, expose routing mode and evidence to the user, and provide maintenance support for document updates and graph editing. From a software-engineering perspective, the objective is not merely to demonstrate a chatbot, but to create a maintainable local-first system integrating retrieval, graph querying, interface transparency, and operational workflows within a single full-stack application.

The objectives therefore span both algorithmic and operational layers. At the algorithmic level, the system should retrieve relevant evidence precisely and reduce hallucination. At the operational level, it should remain usable, inspectable, and extensible for real academic data. This dual emphasis is important because an institutional assistant must be reliable in deployment, not only persuasive in demonstration.

1.7 SCOPE AND DELIMITATIONS

The scope of NexRag is limited to institutional academic assistance within a bounded educational environment. The system is designed to answer questions related to regulations, course information, faculty roles, seminar guidance, project formatting, and similar academic topics. Its knowledge sources consist of PDF documents and RDF/Turtle graph files, and its deployment model assumes locally available backend, retrieval, graph, and language-model services.

The project is not intended to serve as a universal academic search engine or a general-purpose open-domain assistant. Its performance depends on the quality and coverage of the indexed documents and graph content. The current router is rule-based rather than learned, and the graph schema is intentionally compact. During the verification session documented later, the Fuseki service was not active, so graph-backed answering could not be demonstrated end to end. These delimitations

do not reduce the architectural validity of the project, but they define the present maturity level of the implementation.

The scope also excludes automated institution-wide data ingestion from legacy administrative systems. At the current stage, documents and graph content are maintained through controlled files and update workflows. This keeps the project tractable and transparent, while leaving room for future automation once the core hybrid architecture is stabilized.

1.8 ORGANIZATION OF REPORT

This report is organized into five chapters excluding the front matter and references. Chapter 1 introduces the domain, motivation, challenges, objectives, and scope of the work. Chapter 2 reviews the literature underlying retrieval-augmented generation, dense and sparse retrieval, reranking, knowledge graphs, and evaluation. Chapter 3 presents the proposed NexRag system, including its architecture, modules, algorithms, and design decisions. Chapter 4 discusses implementation evidence, verification results, retrieval behavior, and observed operating characteristics. Chapter 5 concludes the report and outlines the future scope of the system.

CHAPTER 2: LITERATURE REVIEW

2.1 INTRODUCTION

The design of NexRag is informed by research in retrieval-augmented generation, dense retrieval, lexical ranking, reranking, knowledge graphs, and RAG evaluation. These areas collectively address a central problem: a conversational system becomes useful in institutional settings only when it can retrieve external evidence, preserve relational accuracy, and present grounded answers. The literature therefore suggests a movement away from purely generative chatbots toward hybrid systems that combine retrieval, ranking, and structured knowledge access (Lewis et al., 2020; Gao et al., 2023).

This chapter reviews the main works most relevant to the proposed system. For each work, the emphasis is on the problem addressed, the method introduced, the key result or contribution, and its relevance to NexRag. The discussion shows that no single prior system fully matches the needs of a local academic assistant, but the literature clearly supports a hybrid architecture combining graph querying, document retrieval, reranking, and grounded response generation.

2.2 REVIEW OF RELATED WORKS

2.2.1 REALM: RETRIEVAL-AUGMENTED LANGUAGE MODEL PRE-TRAINING

REALM, proposed by Guu et al. (2020), demonstrated that retrieval can be incorporated into language-model learning rather than being treated as an external post-processing step. The work addressed knowledge-intensive question answering by allowing the model to access a large evidence store during training and inference. Its main result was that retrieval-aware pre-training improved factual performance compared with parametric-only approaches. For NexRag, REALM is important mainly as a conceptual shift: it established retrieval as a core component of language systems. Its limitation is practical complexity, since pre-training a retrieval-aware model is beyond the scope of a lightweight institutional deployment.

2.2.2 DENSE PASSAGE RETRIEVAL FOR OPEN-DOMAIN QUESTION ANSWERING

Dense Passage Retrieval (DPR) by Karpukhin et al. (2020) addressed the problem of semantic mismatch between user questions and evidence passages. Using dual encoders, DPR placed queries and passages in a shared vector space and showed strong open-domain QA performance, often surpassing BM25 on benchmark datasets. The relevance to NexRag is direct, because institutional queries frequently differ from the exact wording used in official documents. DPR supports the use of embedding-based retrieval in the proposed system. Its limitation is that semantically similar passages are not always institutionally authoritative, which is why NexRag complements dense search with lexical retrieval and reranking.

2.2.3 RETRIEVAL-AUGMENTED GENERATION

Lewis et al. (2020) introduced the RAG framework, which couples external retrieval with neural generation. The method addressed hallucination and stale knowledge by grounding responses in retrieved documents. The major contribution of this work was to provide a clear architecture for knowledge-intensive language generation and to show that factuality improves when the generator conditions on retrieved evidence. NexRag adopts this principle directly, but extends it by adding a knowledge-graph pathway for structured institutional facts. Classical RAG is therefore the conceptual foundation of the project, although it does not by itself solve relational question answering.

2.2.4 FUSION-IN-DECODER AND MULTI-PASSAGE REASONING

Izacard and Grave (2021) showed that open-domain QA improves when the model can combine evidence from multiple retrieved passages instead of relying on a single best hit. Their Fusion-in-Decoder approach highlighted the value of aggregating complementary evidence across chunks. This insight is relevant to NexRag because academic answers often depend on more than one passage, especially when guidelines are distributed across multiple pages or documents. The work supports multi-chunk context construction in the proposed system. Its limitation for the present project is computational cost, since full end-to-end decoder fusion is heavier than prompt-based evidence composition.

2.2.5 SENTENCE-BERT FOR EFFICIENT SEMANTIC RETRIEVAL

Sentence-BERT (SBERT), proposed by Reimers and Gurevych (2019), solved a practical efficiency problem in semantic similarity by producing reusable sentence embeddings suitable for vector search. The method enabled fast comparison of many text segments using cosine similarity and became foundational for semantic search applications. NexRag uses this line of development through a sentence-transformer embedding model for chunk indexing and query encoding. The main relevance of SBERT is operational efficiency for local semantic retrieval. Its limitation is that embedding quality depends on the pretrained model and may not fully capture institution-specific jargon.

2.2.6 BM25 AND PROBABILISTIC LEXICAL RANKING

Robertson and Zaragoza (2009) formalized the probabilistic relevance framework underlying BM25, one of the strongest and most durable lexical ranking methods in information retrieval. BM25 addresses exact term importance through term frequency, inverse document frequency, and document-length normalization. Its continuing relevance lies in domains where exact terms, codes, and formal policy language matter. In NexRag, BM25 complements dense retrieval by capturing document language that should not be diluted by semantic smoothing. Its limitation is weak handling of paraphrase, which is why it is most effective when combined with dense retrieval rather than used alone.

2.2.7 PASSAGE RE-RANKING WITH BERT

Nogueira and Cho (2019) showed that BERT-based cross-encoders can substantially improve passage ranking by jointly modeling query-passage relevance. The work addressed the gap between broad candidate recall and precise top-result selection, and its main contribution was a highly accurate reranking stage for passage retrieval. This idea is implemented in NexRag through a cross-encoder reranker applied after dense and sparse candidate generation. The relevance is clear: academic assistants require a small set of highly reliable supporting passages. The main limitation is latency, which restricts reranking to a short candidate list.

2.2.8 RANK FUSION FOR HYBRID RETRIEVAL

Cormack, Clarke and Buttcher (2009) showed that Reciprocal Rank Fusion

(RRF) is a robust method for combining ranked lists from heterogeneous retrieval systems. The importance of this work lies in its simplicity: it does not require score normalization across fundamentally different retrievers, yet it can outperform more complex voting-style approaches. For systems such as NexRag, which combine dense and sparse retrieval, this literature is directly relevant because each retriever captures different evidence signals. Dense retrieval improves semantic recall, whereas BM25 preserves exact institutional phrasing. The main limitation is that rank fusion alone does not replace deeper relevance modeling, which is why it is best used before a neural reranking stage rather than as a final decision rule.

2.2.9 KNOWLEDGE GRAPHS FOR STRUCTURED REASONING

Hogan et al. (2021) provided a comprehensive account of knowledge graphs, showing how explicit entities and relations support structured querying, semantic clarity, and explainability. This work is important because many academic questions are inherently relational, involving links among faculty, courses, departments, and regulations. For NexRag, the knowledge-graph literature justifies the use of RDF, Turtle, and SPARQL for fact-oriented queries that are not naturally solved by text similarity alone. Its limitation is the need for schema design and data maintenance, which creates an ongoing update burden.

2.2.10 GRAPH-AUGMENTED QUESTION ANSWERING

Recent work has explored how graph evidence can be integrated with language models and retrieval pipelines. KAPING uses retrieved knowledge-graph facts to augment prompts for zero-shot graph question answering (Baek, Aji and Saffari, 2023). REANO strengthens retrieval-augmented readers through graph generation over retrieved evidence (Fang, Meng and Macdonald, 2024). G-Retriever focuses on retrieval over textual graph structures for graph-based question answering (He et al., 2024). These systems address a common problem: graph structure improves factual control, but it must be connected to language generation in a scalable way. Their collective relevance to NexRag lies in validating a hybrid graph-text pathway. The main limitation is implementation complexity, which is higher than what is practical for a lightweight B.Tech deployment.

2.2.11 EVALUATION AND BENCHMARKING FRAMEWORKS

Benchmark and evaluation literature also shapes the design of NexRag. BEIR demonstrated that retrieval effectiveness varies sharply across domains and that zero-shot performance on one dataset does not guarantee robustness elsewhere (Thakur et al., 2021). Gao et al. (2023) surveyed RAG systems and clarified the major design dimensions of retrievers, generators, knowledge sources, and evaluation strategies. RAGAs proposed multi-dimensional evaluation of retrieval-augmented generation through measures such as context precision, answer relevance, and faithfulness (Es et al., 2024). Together, these works show that hybrid assistants should be evaluated not only by fluency, but by retrieval quality, evidence faithfulness, and domain suitability. This is directly relevant to the repository-based evaluation strategy adopted in the present project.

2.3 COMPARATIVE ANALYSIS

Table 2.1 compares the principal works reviewed in this chapter and highlights their relevance to NexRag.

Table 2.1: Comparative Review of Related Research Works

Work	**Problem / Method**	**Major Strength**	**Key Limitation**	**Relevance to NexRag**
Guu et al. (2020) REALM	Retrieval-aware LM pre-training	Strong knowledge integration	High training cost	Motivates retrieval-centric design
Karpukhin et al. (2020) DPR	Dense dual-encoder retrieval	Handles semantic mismatch	May miss exact policy phrasing	Supports dense search layer
Lewis et al. (2020) RAG	Retrieval-grounded generation	Reduces hallucination	Limited structured reasoning	Core architectural principle
Izacard and Grave (2021) FiD	Multi-passage fusion for QA	Aggregates distributed evidence	Resource-intensive	Supports multi-chunk context

Reimers and Gurevych (2019) SBERT	Efficient semantic embeddings	Practical vector retrieval	Domain gap risk	Enables embedding-based indexing
Robertson and Zaragoza (2009) BM25	Probabilistic lexical ranking	Strong exact-term matching	Weak paraphrase handling	Supports sparse retrieval
Nogueira and Cho (2019)	Cross-encoder reranking	High precision ranking	Higher latency	Supports candidate reranking
Cormack et al. (2009)	Reciprocal Rank Fusion	Robust hybrid ranking combination	Requires downstream filtering	Motivates future dense-sparse fusion
Hogan et al. (2021)	Knowledge graph foundations	Explicit relational semantics	Requires curation	Supports graph module design
Baek et al. (2023), Fang et al. (2024), He et al. (2024)	Graph-augmented QA methods	Improve structured grounding	Complex pipelines	Validates hybrid graph-text path
Thakur et al. (2021), Gao et al. (2023), Es et al. (2024)	Benchmarking, taxonomy, RAG evaluation	Better diagnostic understanding	No direct institutional benchmark	Guides evaluation strategy

The comparison shows that no individual method fully addresses the requirements of a local institutional academic assistant. Dense retrieval is effective for semantic access to documents, but it is not ideal for explicit relations. Knowledge graphs provide precise structured facts, but they do not inherently deliver natural language explanation. RAG improves grounded answer generation, but its quality depends on the retriever and the relevance of the evidence. Reranking and evaluation frameworks improve precision and diagnosis, but they do not replace the need for a well-designed hybrid architecture. NexRag therefore emerges as a synthesis of complementary methods rather than a direct adoption of any single prior system.

The literature also shows a progression from isolated component optimization toward end-to-end evidence pipelines. Early work often evaluated retrievers,

generators, or graphs separately, whereas recent systems increasingly treat retrieval, grounding, and explanation as a coupled design problem. This shift is significant for the present project because academic assistance is especially sensitive to failures at the interface between components. A retriever may surface relevant text, yet the final answer can still be misleading if the evidence is poorly fused or insufficiently attributed. The proposed system therefore draws not only on individual algorithms, but also on this broader systems-oriented view of grounded question answering.

2.4 RESEARCH GAPS

The reviewed literature reveals four main gaps that justify the proposed system. First, most RAG and retrieval studies focus on open-domain benchmarks rather than bounded institutional environments. Academic institutions require support for procedural documents, faculty-course relations, and department-specific rules, which differ from encyclopedic or web-scale corpora. Second, relatively few systems combine graph reasoning and document retrieval in a lightweight local deployment intended for practical operation within a college environment.

Third, the literature pays less attention to user-facing transparency than to retrieval or generation accuracy. In academic assistance, users benefit from knowing whether an answer came from a graph fact, a document passage, or a combined evidence path. Fourth, many research systems underemphasize update workflows for documents and structured data, even though maintainability is essential in institutional use. NexRag is designed to address these gaps by combining hybrid retrieval, graph querying, local deployment, evidence visibility, and maintainable update paths within one system.

An additional gap concerns operational reliability in constrained environments. Many published systems assume always-on services, abundant compute, or centralized managed infrastructure. In contrast, educational institutions often require lower-cost deployments that can continue operating with local resources and tolerate partial service unavailability. This makes graceful degradation, modular maintenance, and bounded-scope deployment central design concerns. The literature offers strong algorithmic ideas, but comparatively fewer institution-ready implementations that integrate those ideas into a practical local workflow. NexRag is proposed precisely in response to this gap.

CHAPTER 3: PROPOSED SYSTEM

3.1 SYSTEM OVERVIEW

NexRag is a local-first academic assistant designed to answer institutional queries through a hybrid evidence pipeline. The system combines three complementary components: a knowledge graph for structured institutional facts, a retrieval engine for unstructured academic documents, and a local language model for grounded response generation. This design is motivated by the observation that academic questions are heterogeneous. Queries such as "Who teaches Computer Graphics?" are relational and are best answered from structured data, whereas queries such as "What are the seminar guidelines?" depend on narrative explanations embedded in PDF documents.

At runtime, the system accepts a user query through the frontend, forwards it to a FastAPI backend, classifies the query intent, retrieves evidence from the knowledge graph and/or document index, and then constructs a grounded prompt for a local language model served through Ollama. The final response is returned together with supporting evidence, routing mode, and a confidence estimate. The architecture therefore treats answer generation as the final stage of an evidence-selection process rather than as an isolated generative task.

The repository confirms a modular implementation. The API layer coordinates requests, the router performs lightweight intent detection, the graph module executes SPARQL-based retrieval, the retrieval module manages PDF extraction and ranking, and the LLM module produces the final response. The frontend complements this backend pipeline with source inspection, system-status display, and graph-editing support, making the system usable as an operational academic assistant rather than a backend-only prototype.

At a design level, NexRag treats question answering as constrained evidence synthesis. The system does not begin from the assumption that the model already "knows" the answer; instead, it treats the model as a formatter and explainer operating over retrieved institutional evidence. This distinction is critical in an educational

context because it makes answer quality depend primarily on the correctness of retrieved graph facts and passages rather than on opaque parametric memory. The architecture is therefore intentionally conservative: retrieval and evidence selection occur first, while generation is used to organize and explain already selected content.

Table 3.1: Core Software Stack Used in NexRag

Layer	**Technology**	**Role**
Frontend	React, Vite, TypeScript	Query input, chat view, sources, status, KG editor
Backend API	FastAPI	Request orchestration and service endpoints
Dense retrieval	Sentence Transformers, FAISS	Embedding generation and vector search
Sparse retrieval	BM25 (`rank_bm25`)	Exact-term and lexical search
Reranking	CrossEncoder (`ms-marco-MiniLM-L-6-v2`)	Precision ranking of shortlisted chunks
Knowledge graph	RDF/Turtle, Fuseki, SPARQL	Structured institutional fact retrieval
Generation	Ollama, `llama3.2:3b`	Local grounded answer generation

3.2 SYSTEM ARCHITECTURE

The architecture follows a layered hybrid-processing model. The frontend collects user questions and displays responses with route and source information. The backend receives the request and invokes a router that categorizes the query into coarse academic intents such as faculty information, course information, regulations, definitions, or general queries. This routing step does not produce the answer by itself; instead, it decides which evidence pathways should be prioritized.

The first evidence pathway is the knowledge graph. When the query is relational or entity-centric, the backend issues SPARQL queries over RDF data stored in Turtle files and served through Apache Jena Fuseki. The second pathway is the document-retrieval engine. Academic PDFs are chunked, indexed, and searched using both dense vector similarity and BM25 lexical ranking. The candidate passages from both paths are reranked and filtered before being passed to the language model. The system can therefore operate in graph mode, vector mode, combined mode, or no-hit mode depending on evidence availability.

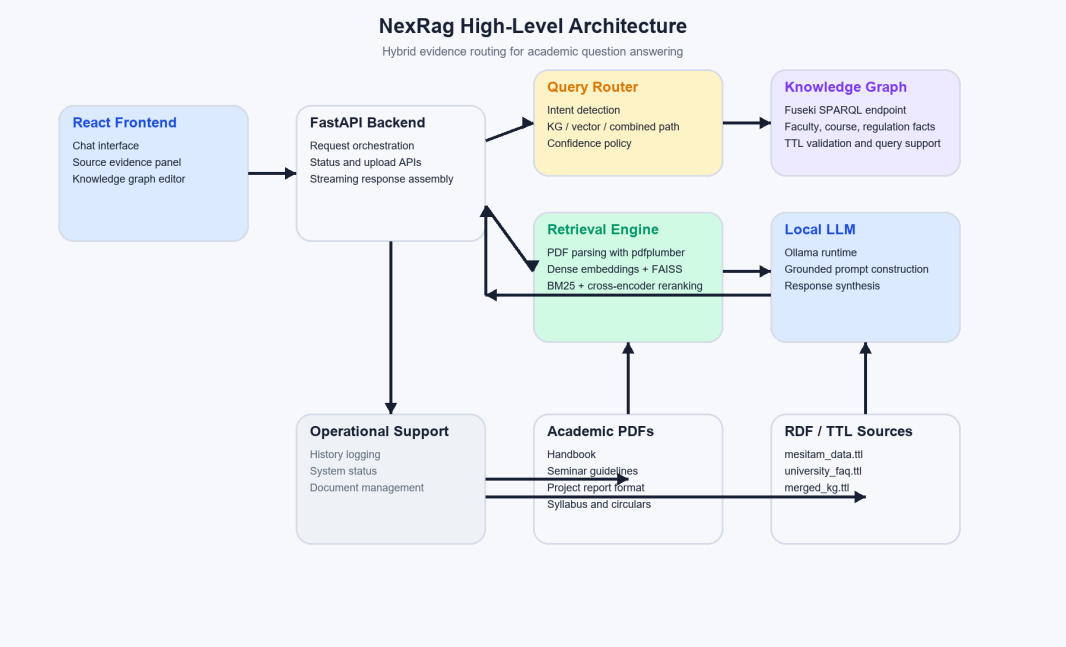

Fig3.1. High-Level Architecture of NexRag

The final step in the architecture is grounded response generation. Retrieved graph facts and top-ranked document snippets are composed into a prompt that instructs the local model to answer from available evidence and avoid unsupported claims. The backend then returns the generated answer, the route label, supporting sources, and a confidence value to the frontend. This architecture improves transparency because users can inspect not only the final answer, but also the evidence path used to produce it.

The architecture also supports controlled degradation. If graph evidence is unavailable, the system can still respond through document retrieval; if retrieval

is weak, the user can be shown reduced confidence and limited evidence instead of an artificially certain answer. This behavior is especially important in local deployments, where auxiliary services such as a graph endpoint or model runtime may not always be active simultaneously. By separating routing, retrieval, generation, and status inspection, NexRag remains interpretable even when operating in a reduced-capability state.

Fig3.2. End-to-End Query Processing Flow

3.2.1 ARCHITECTURAL RATIONALE

The selected architecture is based on complementarity. A graph-only design would provide precise relations but weak explanatory detail, while a document-only design would provide narrative evidence but weaker support for structured entity questions. A generative-only system would remain vulnerable to hallucination. NexRag therefore integrates graph querying, hybrid retrieval, and local generation in a modular manner so that each component addresses a distinct weakness of the others. The modular design also improves maintainability, because routing logic, graph schema, retrieval models, and language-model configuration can evolve independently.

Another reason for this architecture is lifecycle manageability. Institutional documents change periodically, faculty assignments may be revised each semester, and new procedural guidelines may be introduced without notice. A tightly coupled

system would make such updates difficult to isolate and validate. The modular architecture avoids that problem by allowing PDF re-indexing, graph updates, and prompt tuning to be performed as bounded maintenance actions. This improves the long-term practicality of the system beyond the initial project demonstration.

3.3 MODULES

The implementation is organized into functional modules that coordinate through the API layer. Each module addresses a specific operational responsibility, which makes the system easier to test, extend, and maintain.

Table 3.2: Major Backend Modules and Responsibilities

Module	**Primary File**	**Responsibility**
API orchestration	`api.py`	Request handling, response assembly, service endpoints
Query routing	`router.py`	Coarse intent classification
Knowledge graph service	`kg.py`	SPARQL query execution and TTL validation
Retrieval engine	`rag.py`	PDF extraction, chunking, indexing, retrieval, reranking
Generation engine	`llm.py`	Grounded prompt construction and answer generation
Logging and support	`logger.py`, `merge_kg.py`	Query history and graph-merge workflows

3.3.1 USER INTERFACE MODULE

The user interface is implemented with React and Vite and provides three core views: the conversation interface, an intelligence panel, and a knowledge-graph

editor. The conversation interface accepts natural-language queries and renders streaming responses with route labels and source summaries. The intelligence panel exposes retrieved documents, graph evidence, model identity, and service readiness. The graph editor allows controlled maintenance of Turtle files through the same interface. This design is important because it transforms the assistant from a hidden pipeline into an explainable and maintainable user-facing system.

3.3.2 API ORCHESTRATION MODULE

The API module serves as the control layer of the application. It exposes endpoints for status checks, document upload and deletion, chat requests, history retrieval, graph file access, and graph restart workflows. More importantly, it assembles the final chat payload by combining route output, graph evidence, vector evidence, and confidence scoring. Centralizing this logic in `api.py` prevents duplication and ensures that the system behaves consistently across standard and streaming chat modes.

3.3.3 QUERY ROUTING MODULE

The routing module performs lightweight intent classification using keyword-driven heuristics. It identifies categories such as faculty information, course information, regulation, comparison, definition, and general queries. This approach is suitable for a bounded academic domain because common query types are predictable and the rules are easy to maintain. The trade-off is reduced flexibility for ambiguous or highly paraphrased inputs, which is recognized as a future enhancement area.

The router is intentionally coarse-grained rather than over-engineered. Its purpose is not to produce a final semantic label with research-grade accuracy, but to guide the first evidence decision with minimal latency and maximum inspectability. In a bounded academic corpus, this is a practical design choice because many high-frequency questions are repetitive and lexically stable. The router can therefore serve as a lightweight control mechanism while leaving the final relevance decision to retrieval and reranking.

3.3.4 KNOWLEDGE GRAPH MODULE

The knowledge graph module handles structured institutional facts stored as

RDF/Turtle data and queried through SPARQL. It extracts search terms from user queries, matches them to faculty, course, department, or regulation entities, and formats the returned bindings into readable answers. The same module also supports TTL validation and update inspection, warning when an edit removes triples. This makes the graph service both a fact-retrieval subsystem and a controlled data-maintenance layer.

The graph path is particularly important for questions where exact relations matter more than descriptive explanation. Queries about who handles a course, which department offers a subject, or how an entity is connected within the institution benefit from explicit triples rather than paragraph-level similarity. By storing such relations in RDF form, the system separates authoritative institutional facts from less structured descriptive text. This improves explainability because graph answers can be traced back to explicit entities and predicates rather than inferred from loosely related passages.

3.3.5 RETRIEVAL ENGINE MODULE

The retrieval engine is responsible for processing academic PDFs and producing ranked evidence passages. It extracts text with `pdfplumber`, splits text into overlapping chunks, encodes chunks using a sentence-transformer model, stores the vectors in FAISS, and keeps associated metadata for source tracking. At query time it performs both dense retrieval and BM25 search, merges candidate sets, reranks them using a cross-encoder, and filters weak results. This layered design balances semantic recall, lexical precision, and ranking quality.

The chunking and metadata strategy is central to retrieval quality. Each chunk preserves document identity, page-level provenance, and contextual association with neighbouring text so that retrieved evidence remains interpretable when shown to the user. This is necessary because academic rules are often expressed across consecutive clauses, tables, or page sections rather than in isolated sentences. By retaining source metadata throughout the pipeline, NexRag can present answers together with document names, page references, and relevance signals, thereby improving both utility and trust.

3.3.6 LOCAL LANGUAGE MODEL MODULE

The generation module constructs a grounded prompt that includes the user query, the selected evidence, and instructions to answer conservatively when evidence is insufficient. The system uses a locally served language model through Ollama, which supports privacy-aware and cost-controlled deployment. The generated answer is streamed back to the interface, improving perceived responsiveness without changing the underlying evidence pipeline.

Prompt construction is deliberately evidence-first. Graph facts and document snippets are serialized into a compact context block, and the model is instructed to prefer explicit evidence, avoid unsupported assumptions, and acknowledge insufficiency when necessary. This makes the LLM function as a controlled response synthesizer instead of an unrestricted answer generator. In a college setting, such bounded prompting is preferable because the primary goal is reliability of institutional guidance rather than stylistic elaboration.

3.3.7 LOGGING, HISTORY, AND MAINTENANCE MODULE

The system also includes operational support components. Query logs are written to a CSV history file, status endpoints expose document and vector-index counts, and the graph workflow supports merge and restart operations. These components do not define the intelligence of the system, but they are essential for maintainability and evaluation because they make runtime behavior inspectable and auditable.

3.4 ALGORITHMS USED

The intelligence of NexRag depends on a sequence of algorithms rather than a single model. The main computational stages are embedding generation, vector search, lexical ranking, neural reranking, confidence computation, and final evidence-guided generation.

3.4.1 EMBEDDING TECHNIQUE

The document corpus is divided into chunks and embedded using a sentence-transformer model. If `x_i` denotes the `i`-th text chunk and `f(.)` denotes the embedding model, each chunk vector is computed as:

$$e_i = f(x_i) \tag{3.1}$$

The same encoder is used to generate the query embedding. This representation allows semantically related questions and passages to be compared even when they do not share exact vocabulary. Embedding-based retrieval is therefore valuable for academic queries phrased in natural language rather than formal handbook language.

3.4.2 DENSE VECTOR SEARCH

FAISS is used for first-stage dense retrieval. Given a query embedding `q` and a candidate chunk embedding `x`, the current implementation uses Euclidean distance to identify nearby candidates:

$$d(q, x) = \|q - x\|_2 \tag{3.2}$$

Lower distance implies greater similarity in the embedding space. This search is efficient and suitable for first-stage recall, but it is not sufficient by itself because semantically related passages are not always the most authoritative or precise for an institutional query.

3.4.3 COSINE SIMILARITY AS A STANDARD SEMANTIC MEASURE

Cosine similarity is the standard theoretical measure for comparing embedding direction and is included here because it explains semantic relevance more intuitively:

$$\cos(q, x) = \frac{q \cdot x}{\|q\|\|x\|} \tag{3.3}$$

Although the deployed FAISS index uses L2 distance, cosine similarity remains conceptually important because both measures serve the same goal of capturing semantic proximity between a query and candidate passages.

3.4.4 BM25 SPARSE RETRIEVAL

To preserve lexical precision, NexRag uses BM25 over tokenized chunk text. For a document or chunk `D` and query `Q`, the BM25 score is:

$$\begin{aligned} \text{BM25}(D, Q) = \sum \text{IDF}(q_i) \cdot \\ \frac{f(q_i, D)(k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b \frac{|D|}{\text{avgdl}}\right)} \end{aligned} \tag{3.4}$$

This stage is particularly useful for exact rule names, course codes, abbreviations, and formal policy phrases. Dense retrieval and BM25 therefore serve different but complementary purposes within the same pipeline.

3.4.5 CROSS-ENCODER RERANKING

After dense and sparse retrieval, the shortlisted candidates are reranked by a cross-encoder that jointly scores the query and passage. The raw score z is normalized through a sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{3.5}$$

The normalized value is treated as a relevance score for sorting and thresholding. This stage improves the final quality of the supporting evidence shown to the user.

3.4.6 CONFIDENCE COMPUTATION

The backend estimates confidence from the available evidence. Graph evidence, vector evidence, and corroboration between the two are combined heuristically into a user-facing percentage. The resulting value is not a calibrated probability, but it provides a practical signal of evidence strength and is preferable to silent overconfidence in an academic assistant.

In operational terms, confidence is better understood as a retrieval-strength indicator than as a probabilistic guarantee of correctness. High confidence implies that the returned answer is supported by strong or corroborating evidence, whereas lower confidence suggests weak retrieval, partial graph support, or marginal ranking scores. This distinction is important because it encourages users to interpret the answer together with its evidence rather than treating the confidence value as a replacement for evidence inspection.

3.4.7 PSEUDO-CODE FOR MAJOR ALGORITHMS

Algorithm 3.1 Document Ingestion and Indexing

```
Input: PDF p, category c
Output: Updated vector and lexical indexes
```

1. Extract text from p page by page
2. Split text into overlapping chunks
3. Attach metadata: source, page, category
4. Encode chunks into dense vectors
5. Insert vectors into FAISS
6. Rebuild BM25 over chunk texts
7. Persist index and metadata

Algorithm 3.2 Hybrid Retrieval and Reranking

Input: Query q, top_k
Output: Ranked evidence list

1. Encode q into a dense query vector
2. Retrieve dense candidates from FAISS
3. Score chunks lexically with BM25
4. Merge dense and sparse candidate sets
5. Run cross-encoder on each query-chunk pair
6. Normalize scores and sort in descending order
7. Filter weak candidates and return top_k

Algorithm 3.3 Query Routing and Response Assembly

Input: User query q
Output: Grounded response payload

1. Classify q using the router
2. Query the knowledge graph if the route suggests it
3. Retrieve ranked document evidence
4. Construct grounded context from available evidence
5. Generate answer using the local LLM
6. Attach route, sources, snippets, and confidence
7. Return payload to the frontend

3.4.8 RRF APPLICABILITY

Formal Reciprocal Rank Fusion is not implemented in the current repository, but it remains directly applicable. Since NexRag already generates dense and sparse candidate rankings, RRF could be introduced before reranking as an interpretable score-fusion strategy. The present architecture therefore supports future adoption of RRF without redesigning the entire retrieval pipeline.

3.5 SYSTEM DESIGN

The system design of NexRag can be understood through data design, control-flow design, API workflow, and maintenance design. These four views explain how information is stored, how queries are processed, and how the system remains operational over time.

3.5.1 DATA DESIGN

NexRag uses heterogeneous storage aligned with the nature of the information being handled. Structured institutional facts are stored in RDF/Turtle files and served through Fuseki. Unstructured content remains in PDF form but is mirrored as chunk metadata and dense vectors for retrieval. Query history is written to CSV logs. This design avoids forcing graph facts and document content into the same storage model.

Table 3.3: Conceptual Data Entities and Storage Artefacts

Information Type	Example Content	Storage Form
Structured entities	Faculty, departments, courses, regulations	RDF/Turtle files
Graph service data	Merged institutional graph	Fuseki dataset
Document corpus	Handbooks, seminar and report guidelines	Local PDF directory
Dense index	Chunk vectors	`vector_store.index`
Chunk metadata	Source, page, text, category	`chunks_metadata.pkl`
Interaction history	Question, answer, intent, latency	`logs/query_history.csv`

3.5.2 DATA FLOW DESIGN

The data flow begins with a user query, proceeds through route detection, graph lookup, and/or retrieval, then converges at grounded prompt construction. The answer, along with route and source evidence, is then returned to the user and logged for later analysis. This design makes NexRag a coordination system rather than a single retrieval call.

This flow also supports auditability. Because the route, retrieved evidence, and response metadata are preserved through the backend, the system can later be inspected to determine how a particular answer was produced. Such traceability is valuable in institutional deployments where administrators may need to verify whether an answer reflected graph content, document evidence, or only a weak retrieval fallback.

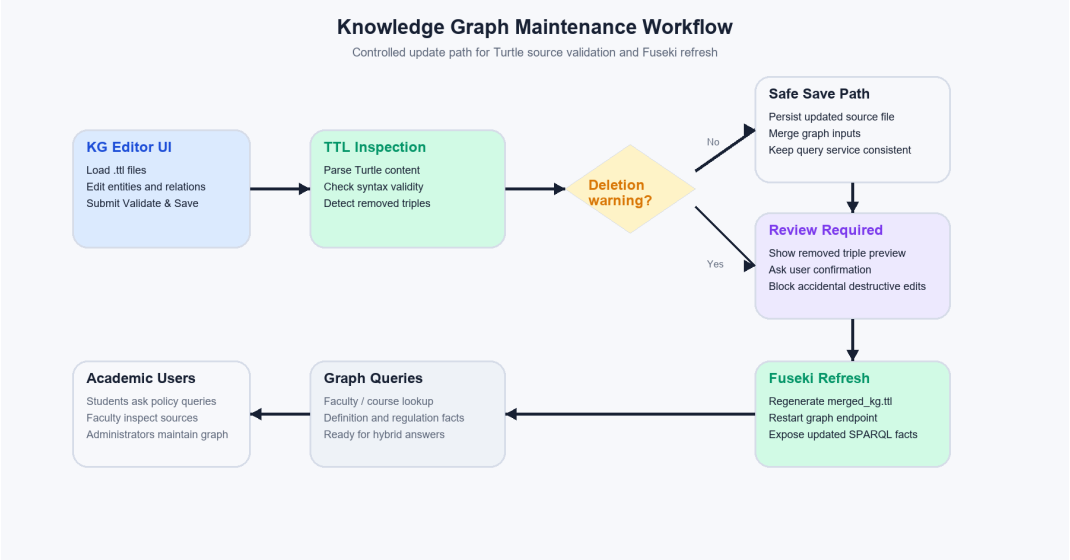

Fig3.3. Knowledge Graph Maintenance and Validation Workflow

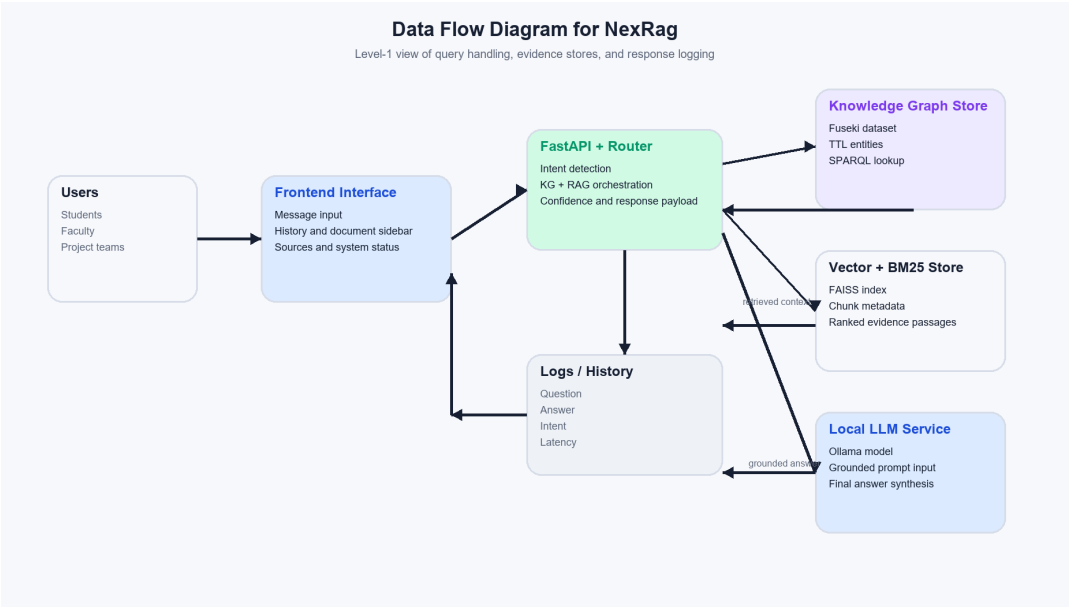

Fig3.4. Data Flow Diagram for the Proposed System

3.5.3 API AND WORKFLOW DESIGN

The backend exposes endpoints for status checks, document management, chat interaction, history access, graph-source editing, and graph restart. This endpoint structure converts the internal architecture into an operational service model and

makes the system easier to verify and maintain.

Table 3.4: API Endpoints and Functional Roles

Endpoint	**Method**	**Function**
`/api/status`	GET	Returns PDF, vector, graph, and model status
`/api/documents`	GET	Lists indexed PDFs
`/api/upload`	POST	Uploads and indexes new documents
`/api/documents/{filename}`	DELETE	Removes a document and rebuilds indexes
`/api/chat`	POST	Returns complete grounded answer
`/api/chat/stream`	POST	Streams grounded answer chunks
`/api/history`	GET	Returns recent interaction history
`/api/kg-files`	GET	Lists editable Turtle files
`/api/kg-source/{filename}`	GET/POST	Reads or updates Turtle source content
`/api/kg-save/{filename}`	POST	Validates and saves graph edits
`/api/kg-restart`	POST	Merges sources and restarts graph service

The main query workflow is straightforward. A question is submitted from the frontend, routed by intent, checked against the graph when appropriate, searched against the vector and BM25 indexes, reranked, and then passed to the local model together with selected evidence. The system finally returns the answer, the evidence

trace, and confidence metadata. This explicit workflow is central to the transparency of the proposed system.

3.5.4 MAINTENANCE DESIGN

The design also incorporates maintenance as a first-class concern. New PDFs can be uploaded and indexed incrementally. Deleted documents trigger vector-store rebuilding to keep retrieval consistent. Knowledge-graph edits are validated before saving, and potentially destructive graph updates are flagged for inspection. These workflows are necessary because academic knowledge sources are revised over time, and an assistant without maintenance support would quickly lose reliability.

By including these maintenance workflows in the core design, the project recognizes that information systems degrade unless their update path is explicit. The practical value of NexRag therefore depends not only on current retrieval performance, but also on how easily new regulations, revised handbooks, or department-specific graph edits can be incorporated without breaking prior functionality. This systems view is one of the major differences between the proposed work and a narrow proof-of-concept chatbot.

3.6 ADVANTAGES

The proposed system offers several advantages over ordinary chatbots and conventional document search. First, it combines structured and unstructured evidence, allowing both relational and descriptive academic questions to be answered within one framework. Second, it reduces hallucination by grounding generation in graph facts and retrieved passages. Third, it improves user trust by exposing route labels, source snippets, and confidence values rather than returning opaque answers.

The system is also operationally suitable for institutional use. Because it is local-first, it avoids mandatory dependence on external APIs and supports privacy-aware deployment. Its modular implementation allows future upgrades to the router, retriever, graph schema, or language model without redesigning the whole application. The presence of upload, logging, and graph-editing workflows further strengthens its maintainability.

3.7 APPLICATIONS

The immediate application of NexRag is institutional academic assistance. Students can use it for attendance rules, seminar and project guidance, faculty-course mapping, and procedural clarifications. Faculty and administrators can use it to reduce repetitive queries and improve the accessibility of official information.

The same architecture is transferable to other bounded domains where structured relations and unstructured documents coexist. Similar systems could support enterprise policy assistance, educational administration, healthcare-training support, or departmental knowledge management. NexRag therefore serves both as a practical institutional tool and as a reusable pattern for local, evidence-aware hybrid assistants.

Within higher education itself, the same design can be extended to multiple administrative layers. Department-level deployments could handle syllabus and faculty information, while institution-level deployments could incorporate examination rules, hostel policies, placement procedures, and accreditation-related knowledge resources. Because the architecture already separates graph facts, document retrieval, and generation, such expansion mainly requires corpus growth and schema extension rather than a complete redesign. This makes NexRag a scalable architectural pattern even if the present implementation remains intentionally bounded.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 INTRODUCTION

This chapter presents the observed implementation evidence for NexRag and discusses the system strictly on the basis of repository artefacts and local verification carried out on **3 April 2026**. The emphasis is on reproducible evidence such as service status, indexed-document counts, test-script output, query logs, and sample retrieval results. This approach avoids unsupported claims while still allowing a meaningful assessment of the system as a hybrid academic assistant.

4.2 IMPLEMENTATION ENVIRONMENT

NexRag is implemented as a local full-stack application. The backend depends on Python, the frontend on the Node-Vite toolchain, and the graph service on Java for Apache Jena Fuseki. The local environment observed during verification is summarized in Table 4.1.

Table 4.1: Repository Verification Summary

Item	Observed Result	Interpretation
Python runtime	3.10.11	Backend requirement satisfied
Node runtime	v24.11.1	Frontend toolchain available
Java runtime	OpenJDK 17.0.17	Compatible with Fuseki
API status snapshot	7 PDFs, 290 chunks, vector ready, local model present	Retrieval pipeline configured
Router test	Passed	Intent rules operational
Retrieval test	Passed model and interface checks	Retrieval subsystem loads correctly
KG test	Passed against live endpoint	Graph retrieval path verified through Fuseki
Frontend build	Passed	Production frontend bundle generated successfully

The observed configuration matches the intended local-first design described in the repository. During local validation, the backend, retrieval stack, graph service, and local model were all available, which allowed both knowledge-graph mode and combined-mode responses to be verified directly from the running application.

4.3 INDEXED KNOWLEDGE SOURCES

The document corpus available in the repository consists of seven PDFs and an indexed chunk store containing 290 chunks. These files represent the knowledge base used by the retrieval engine for academic assistance.

Table 4.2: Current Indexed Document Collection

Sl. No.	Document Name	Observed Role
1	2309.17288v3.pdf	Technical or research reference
2	Circular_TechFest.pdf	Institutional circular content
3	CSE_Syllabus_S5.pdf	Course and syllabus information
4	Guidelines-Format_for_Project_Report.pdf	Project-report formatting guidance
5	KTU_Handbook_2024.pdf	Regulation and academic handbook source
6	SeminarGuidelines.pdf	Seminar process and format guidance
7	TipsforgoodPresentation.pdf	Presentation support material

The repository also contains Turtle files such as mesitam_data.ttl, university_faq.ttl, and merged_kg.ttl. Inspection of these files shows graph entities for departments, faculty, courses, and regulations, confirming that the graph layer is structurally meaningful and actively queryable through the live Fuseki endpoint observed during validation.

The composition of the corpus is itself important for interpreting system behavior. The indexed set is not a random collection of documents; it contains precisely the kinds of artefacts that dominate routine academic queries, including handbook regulations, seminar instructions, project-format guidance, and syllabus files. This alignment between corpus content and user intent explains why the retriever performs well on attendance and seminar-related questions. It also indicates that further gains in answer coverage are likely to come more from corpus expansion and graph enrichment than from major architectural change.

4.4 FUNCTIONAL VERIFICATION

Functional verification was performed using the repository's own test scripts. The import and status tests confirmed successful loading of the backend modules and reported seven PDFs, 290 chunks, vector readiness, and an available local

model. The router test passed its provided cases, indicating that the rule-based intent categories are functioning as intended for the covered examples.

The retrieval test confirmed that the embedding model, reranker, and retrieval interfaces load correctly. The graph service was also reachable during validation, and direct API queries such as "Who teaches Computer Graphics?" and "What is the minimum attendance rule?" returned structured graph evidence together with routed responses. The verification therefore supports both the operational state of the retrieval subsystem and the live readiness of the graph subsystem.

From an engineering standpoint, these outcomes show that the project is beyond the stage of a static code demonstration. The router, retriever, metadata store, and model interface are integrated sufficiently to support verifiable end-to-end query handling on the document side. At the same time, the graph-service dependency remains visible as a real operational condition rather than being hidden. This is a useful result because it reveals the actual deployment boundary of the system and clarifies which components require process-level management in a production-like setup.

4.5 ROUTING AND RETRIEVAL OUTCOMES

Direct query checks were used to observe practical behavior on representative academic questions. Table 4.3 summarizes the route labels associated with three typical inputs.

Table 4.3: Sample Query Routing Outcomes

Query	**Detected Intent**	**Primary Evidence Path**
Who teaches Computer Graphics?	`FACULTY_INFO`	Knowledge graph
What is the minimum attendance rule?	`REGULATION`	Document retrieval
Explain seminar guidelines	`DEFINITION`	Document retrieval

For the query "What is the minimum attendance rule?", the top retrieval result came from `KTU_Handbook_2024.pdf` with a relevance score near **0.9663**. The returned passage explicitly stated the minimum attendance requirement and condonation condition, indicating a strong policy match. For the query "Explain seminar guidelines", the highest-ranked results were drawn from `Guidelines-Format_for_Project_Report.pdf` and `SeminarGuidelines.pdf`, showing that the retriever can locate both closely related procedural and seminar-specific content. The results are summarized in Table 4.4.

Table 4.4: Sample Retrieval Outputs and Relevance Scores

Query	**Top Source**	**Page**	**Score**	**Observation**
What is the minimum attendance rule?	`KTU_Handbook_2024.pdf`	1	0.9663	Strong direct policy match
What is the minimum attendance rule?	`KTU_Handbook_2024.pdf`	N/A	0.2770	Secondary supporting passage
Explain seminar guidelines	`Guidelines-Format_for_Project_Report.pdf`	1	0.9485	High-level procedural guidance
Explain seminar guidelines	`SeminarGuidelines.pdf`	1	0.8702	Seminar-specific evidence
Explain seminar guidelines	`SeminarGuidelines.pdf`	N/A	0.6865	Additional supporting detail

These results indicate that the retrieval engine is effective when the query aligns with the indexed academic corpus. They also show the value of reranking, because related documents compete for top positions in procedural queries.

The sample outputs also illustrate the complementary roles of dense and lexical evidence. The attendance-rule query benefits from exact policy wording, which

naturally favours handbook passages containing formal regulatory language. The seminar-guideline query, by contrast, is broader and therefore returns multiple relevant procedural documents that must be ordered by semantic closeness and contextual usefulness. This difference is consistent with the motivation for combining BM25, vector retrieval, and reranking within a single pipeline.

4.6 INTERFACE AND WORKFLOW

The frontend implementation reveals a user-oriented design. The application provides a welcome state with prompt suggestions, a streaming chat view, route badges, source evidence, a system-status panel, and a knowledge-graph editor. These interface choices are important because they expose the reasoning path of the system instead of presenting the answer as a black box.

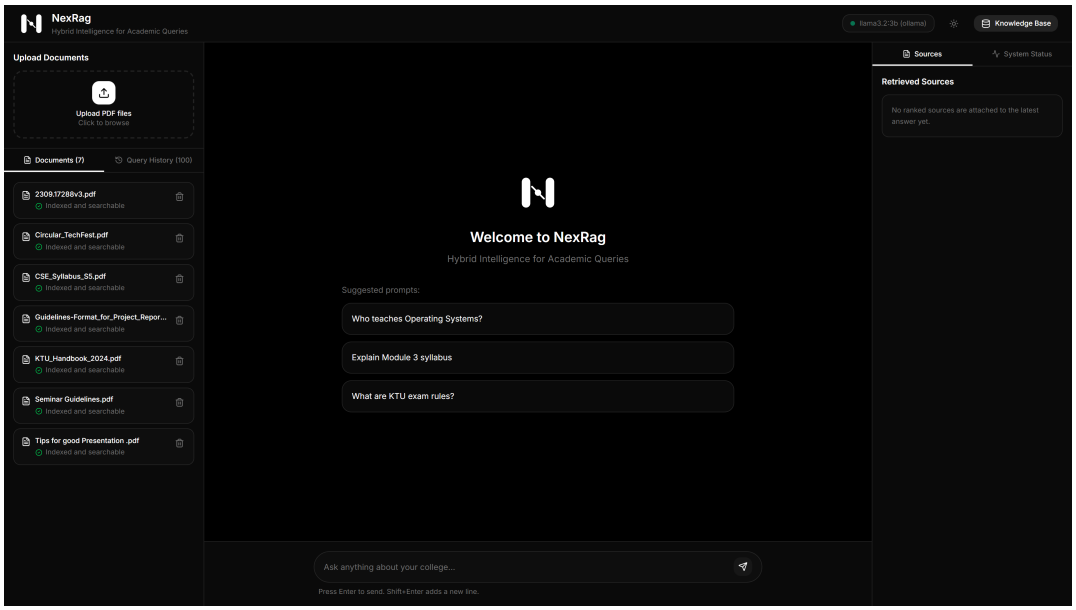

Fig4.1. Main User Interface of the NexRag Frontend

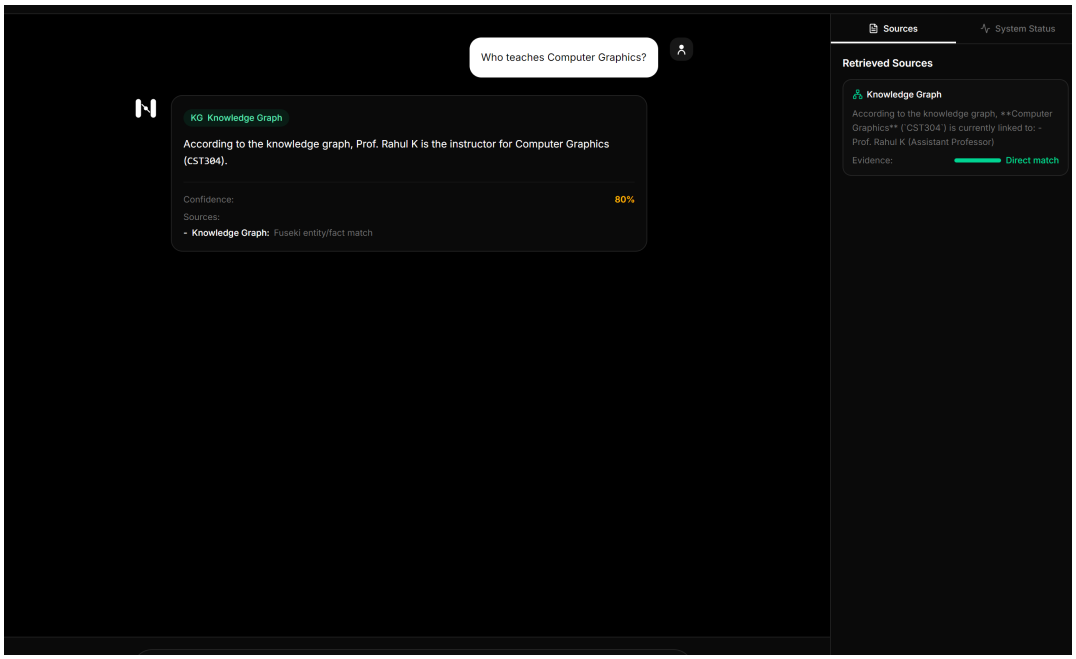

Fig4.2. Streaming Answer View with Routing and Source Evidence

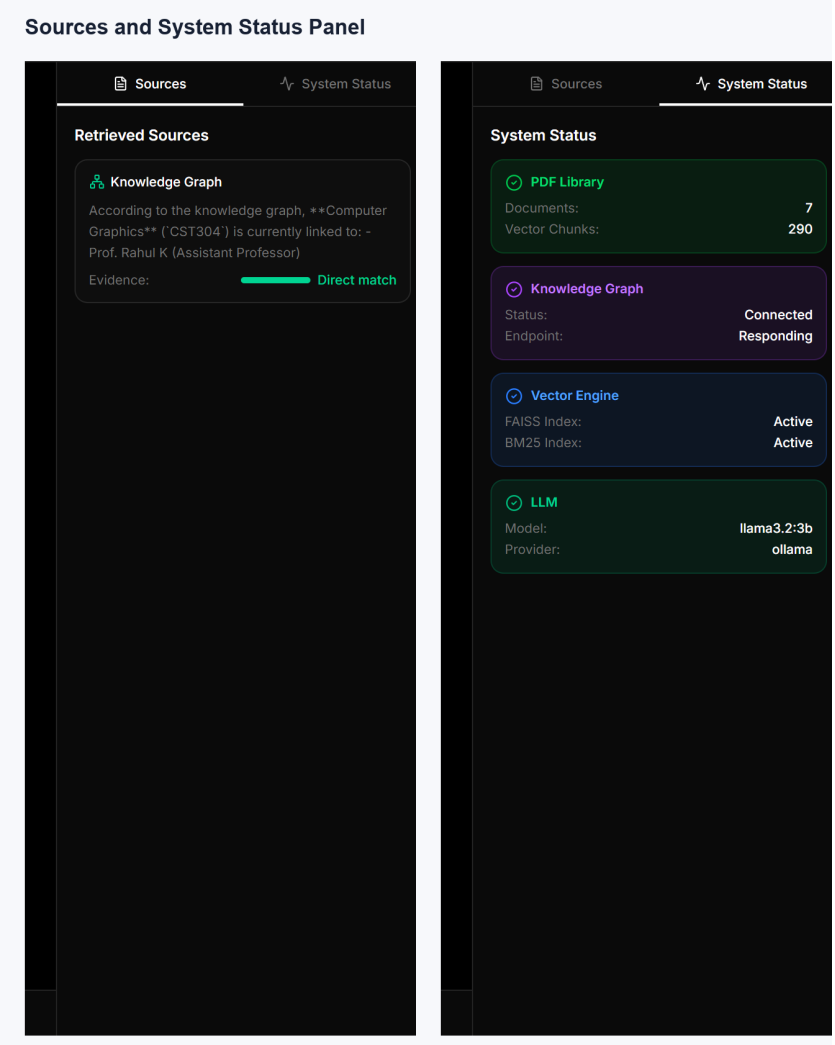

Fig4.3. Intelligence Panel Showing Retrieved Sources and System Status

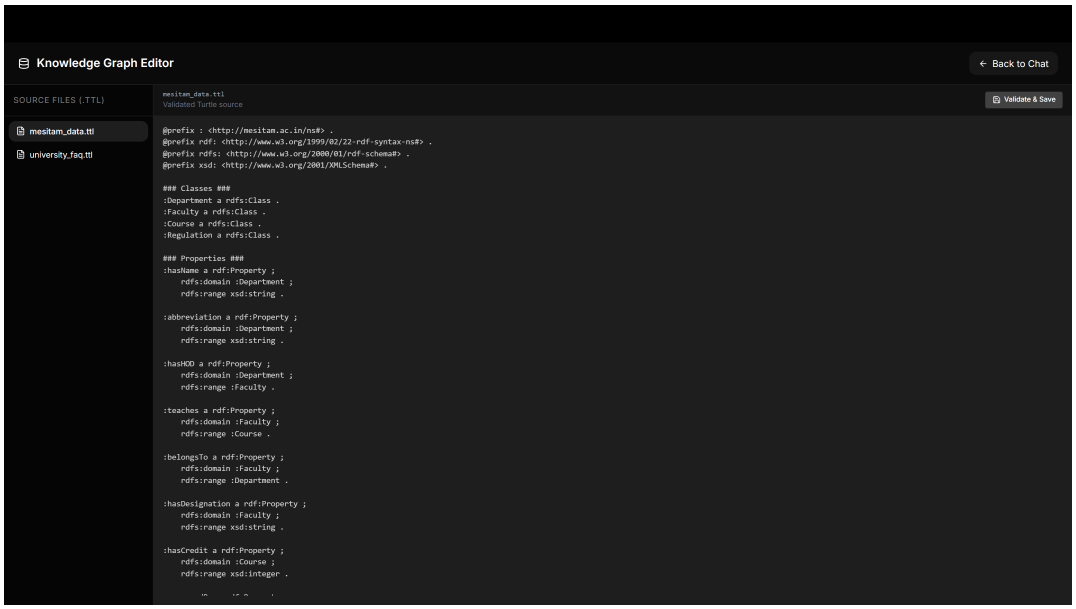

Fig4.4. Knowledge Graph Editor Interface for Turtle File Update

From an evaluation perspective, these interface elements are meaningful because they improve interpretability. Users can determine whether an answer came from graph evidence, document evidence, or a combined route, and can inspect the supporting sources directly.

4.7 PERFORMANCE ANALYSIS

The repository's query-history log provides limited but useful latency observations. Two recorded examples showed response times of **13.87 seconds** for "What is RAG?" and **11.18 seconds** for "Who teaches CS301?". These values are not sufficient for statistical benchmarking, but they are consistent with a local RAG pipeline that includes retrieval and language-model generation.

Table 4.5: Observed Query Log Characteristics

Logged Query	**Intent**	**Latency (s)**	**Observed Response Character**
What is RAG?	DEFINITION	13.87	Explanatory answer from retrieved context
Who teaches CS301?	GENERAL	11.18	Uncertain output due weak evidence at that time

The retrieval results also indicate that the ranking pipeline is functioning well for high-signal queries. The attendance-rule query returned a highly relevant handbook passage, while seminar-related queries retrieved multiple appropriate procedural sources. This suggests that the combined dense, sparse, and reranking strategy is suitable for the current corpus, though larger-scale testing would be needed for stronger conclusions.

These latency observations should be interpreted in relation to the deployment model. NexRag prioritizes local execution, grounded prompting, and source transparency over raw response speed. In a small institutional installation this is an acceptable trade-off because queries are typically low-volume and correctness is more important than sub-second interaction. Nevertheless, the current figures also suggest where optimization effort should be concentrated in future work, particularly in graph-service orchestration, reranking efficiency, and prompt length management.

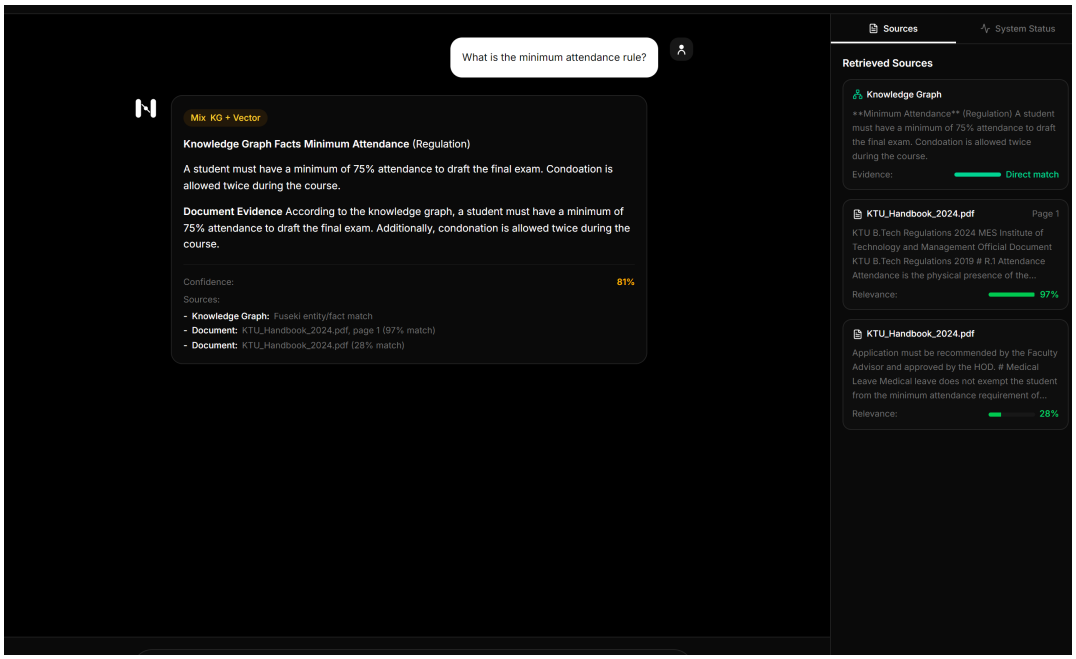

Fig4.5. Retrieval Output for an Attendance Rule Query

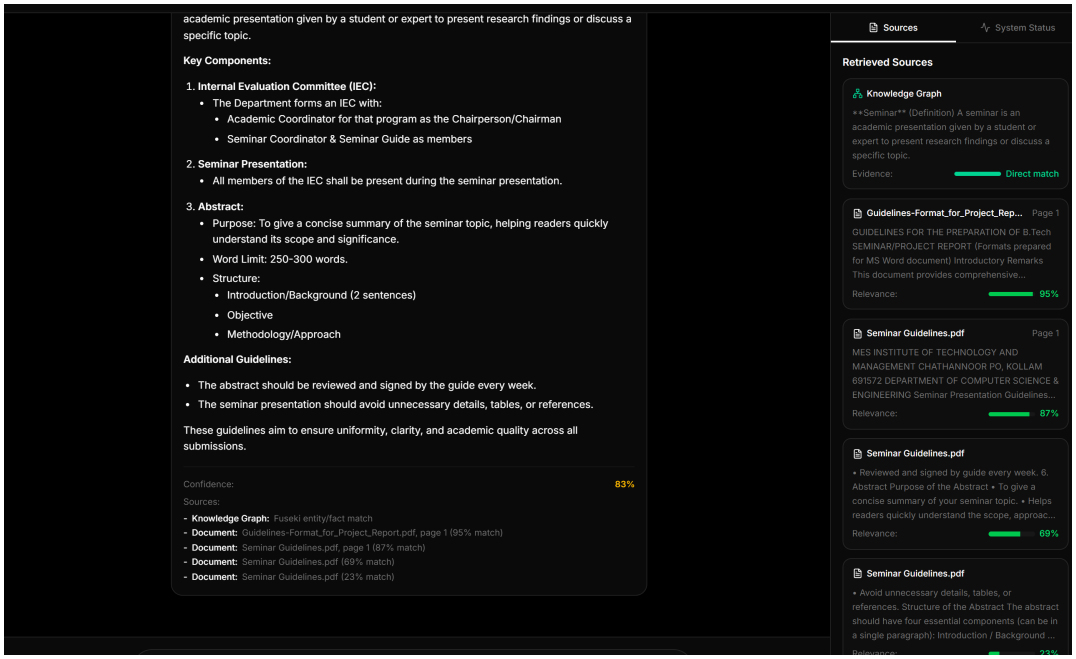

Fig4.6. Retrieval Output for a Seminar Guideline Query

4.8 OPERATING MODES

NexRag can operate in four practical modes depending on evidence availability: knowledge-graph mode, vector-search mode, combined mode, and no-hit mode. These modes are summarized in Table 4.6.

Table 4.6: Comparative Discussion of Operating Modes

Mode	**Trigger Condition**	**Strength**	**Limitation**
Knowledge Graph Mode	Structured fact available	Precise entity-level answer	Limited descriptive depth
Vector Search Mode	Document evidence available	Strong procedural explanation	Weaker explicit relations
Combined Mode	Both graph and document evidence available	Best balance of fact and explanation	Depends on both services
No Retrieval Hit	No reliable supporting evidence	Honest uncertainty handling	No substantive answer

During the observed verification session, the graph service was active, so practical behavior included both pure knowledge-graph answers and combined graph-plus-document responses. This allowed the report to capture live evidence for all three meaningful operating modes other than the no-hit fallback.

This mode-based interpretation is useful because it separates architectural capability from session-specific availability. A hybrid assistant should not be judged only by its best-case path, but by how intelligibly it behaves across changing evidence conditions. NexRag performs reasonably well in this respect because the system can still provide source-backed document answers when the graph service is absent, and the interface is designed to expose that operating condition rather than conceal it.

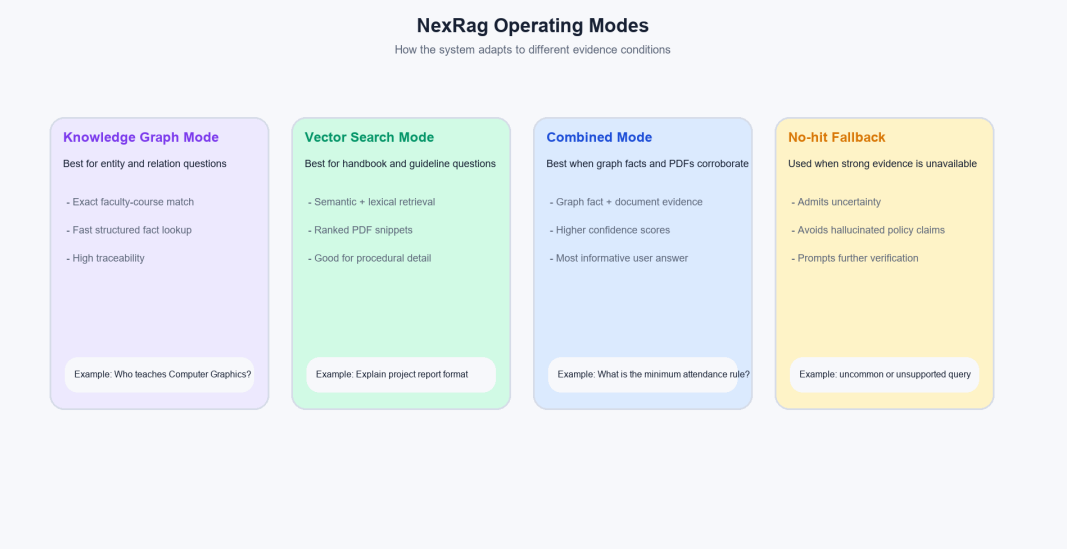

Fig4.7. Comparative Mode Analysis of KG, Vector, and Combined Responses

4.9 EVALUATION LIMITATIONS

The present evaluation has clear limits. Although the live stack was available during validation, the number of representative queries observed was still small, the latency discussion relies on a short interaction history, and the system has not yet been tested through a long-running institutional deployment or a formal user study. In addition, the report evaluates the current local corpus and graph contents, so answer quality will continue to depend on document freshness and graph completeness.

Despite these constraints, the available evidence supports the central claim of the project: NexRag is a coherent hybrid academic assistant with a working document-retrieval pipeline, a structured graph layer, a local generation path, and an interface designed for evidence visibility. The current results therefore demonstrate feasibility and functional readiness, while also indicating the next steps needed for deployment-grade validation.

In summary, the present evaluation is best understood as a functional verification study rather than a full-scale benchmark. Within that scope, the system demonstrates meaningful capability: relevant academic documents are indexed and retrieved, route decisions are produced, evidence is surfaced to the user, and the local-generation path operates over retrieved context. These are strong indicators that the proposed

architecture is technically sound and suitable for continued refinement toward institutional deployment.

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

This report presented NexRag as a local-first hybrid academic assistant that integrates structured knowledge-graph querying, document retrieval, and grounded language generation. The project addressed a common institutional problem: academic information is available, but it is dispersed across regulations, handbooks, course files, and departmental records, making access slow and inconsistent. NexRag responds to this problem by routing queries to the most suitable evidence source and by generating answers only after relevant graph facts and/or document passages have been assembled.

The implemented system demonstrates several important outcomes. First, it shows that hybrid evidence handling is more suitable for institutional question answering than a single-model chatbot. Second, it validates a local deployment model built with FastAPI, React, FAISS, BM25, Fuseki, and Ollama, thereby avoiding compulsory dependence on cloud APIs. Third, it emphasizes evidence transparency through route labels, source traces, confidence indicators, and graph-editing support. Together, these features make the project operationally meaningful as well as technically relevant.

Repository verification further indicates that the retrieval pipeline is functioning in a materially usable state. The current system contains seven indexed PDFs and 290 chunks, the router tests pass, retrieval models load successfully, and representative academic queries return relevant handbook and guideline passages. The graph service was also reachable during local validation, which confirms that structured facts, routed retrieval, and local generation now work together in the observed implementation. The project therefore demonstrates both functional feasibility and a clear path toward fuller institutional deployment.

The work also has value as an engineering study in system integration. It combines heterogeneous knowledge representations, multiple retrieval strategies, local model serving, interface transparency, and maintenance workflows within one

coherent academic application. This integration-oriented contribution is important because many student projects demonstrate isolated models or interfaces, whereas NexRag demonstrates how those components can be assembled into a practical evidence-aware system. At the same time, the project remains appropriately bounded: it does not claim full institutional coverage, but rather establishes a technically sound foundation for that goal.

Table 5.1: Summary of Achievements and Forward Directions

Area	**Current Achievement**	**Future Direction**
Retrieval	Dense, sparse, and reranking pipeline implemented	Improve filtering and score fusion
Knowledge graph	RDF graph and SPARQL workflow prepared	Expand ontology and ensure live service reliability
Generation	Local grounded response generation available	Improve prompting and model selection
Interface	Source-aware chat and status panels implemented	Add role-based and analytical views
Maintainability	Upload, logging, and KG editing workflows present	Automate more administrative tasks
Evaluation	Functional verification and evaluation scripts available	Perform full end-to-end benchmarking

5.2 FUTURE SCOPE

Several extensions can improve NexRag further. The first is stronger intent classification through a learned router or hybrid rule-model approach, which would better handle paraphrase and multi-intent questions. The second is more advanced

score fusion, including formal methods such as Reciprocal Rank Fusion before reranking, especially as the corpus grows. The third is expansion of the knowledge graph to include richer academic entities such as timetables, prerequisites, examination schedules, laboratories, and administrative contacts.

Future work should also focus on rigorous evaluation and operational scaling. The repository already includes the basis for RAGAs-style assessment, and this can be extended into a benchmark comparing graph-only, retrieval-only, and combined configurations. On the deployment side, improved graph-service automation, larger institutional corpora, authentication, role-specific dashboards, and more detailed source analytics would convert the current prototype into a more robust college-wide platform. With these extensions, NexRag can evolve from a strong B.Tech project into a reusable institutional knowledge-assistance system.

Further development should additionally address governance-oriented features that become important in real institutional use. These include controlled update approval for graph edits, versioning of indexed documents, better audit trails for answered queries, and role-aware access to administrative information. Such enhancements would not change the fundamental hybrid architecture, but they would strengthen reliability, accountability, and long-term maintainability. In that sense, the proposed system is not only a completed student project, but also a viable starting point for an institution-grade academic assistance platform.

REFERENCES

1. Apache Software Foundation, *Apache Jena Documentation*, 2026. Available at: https://jena.apache.org/documentation/ (Accessed: 3 April 2026).

2. Baek, J., Aji, A.F. and Saffari, A., "Knowledge-augmented language model prompting for zero-shot knowledge graph question answering," in *Proceedings of the 1st Workshop on Natural Language Reasoning and Structured Explanations*, 2023. Available at: https://aclanthology.org/2023.nlrse-1.7/.

3. Cormack, G.V., Clarke, C.L.A. and Buttcher, S., "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.

4. Es, S., James, J., Espinosa Anke, L. and Schockaert, S., "RAGAs: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024. Available at: https://aclanthology.org/2024.eacl-demo.16/.

5. Fang, J., Meng, Z. and Macdonald, C., "REANO: Optimising retrieval-augmented reader models through knowledge graph generation," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. Available at: https://aclanthology.org/2024.acl-long.115/.

6. Gao, Y. et al., *Retrieval-Augmented Generation for Large Language Models: A Survey*, arXiv:2312.10997, 2023. Available at: https://arxiv.org/abs/2312.10997.

7. Guu, K., Lee, K., Tung, Z., Pasupat, P. and Chang, M.-W., "REALM: Retrieval-augmented language model pre-training," in *Proceedings of the 37th International Conference on Machine Learning*, 2020. Available at: https://proceedings.mlr.press/v119/guu20a.html.

8. He, X., Bresson, X., Hooi, B. and Chawla, N.V., *G-Retriever: Retrieval-Augmented*

Generation for Textual Graph Understanding and Question Answering, arXiv:2402.07630, 2024. Available at: https://arxiv.org/abs/2402.07630.

9. Hogan, A. et al., *Knowledge Graphs*, Morgan and Claypool, 2021.

10. Izacard, G. and Grave, E., "Leveraging passage retrieval with generative models for open domain question answering," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, 2021. Available at: https://arxiv.org/abs/2007.01282.

11. Johnson, J., Douze, M. and Jegou, H., *Billion-scale Similarity Search with GPUs*, arXiv:1702.08734, 2017. Available at: https://arxiv.org/abs/1702.08734.

12. Karpukhin, V. et al., "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020. Available at: https://aclanthology.org/2020.emnlp-main.550/.

13. Lewis, P. et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

14. Nogueira, R. and Cho, K., *Passage Re-ranking with BERT*, arXiv:1901.04085, 2019. Available at: https://arxiv.org/abs/1901.04085.

15. Ollama, *Ollama Documentation*, 2026. Available at: https://ollama.com/ (Accessed: 3 April 2026).

16. Reimers, N. and Gurevych, I., "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992.

17. Robertson, S. and Zaragoza, H., "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

18. Thakur, N., Reimers, N., Ruckle, A., Srivastava, A. and Gurevych, I., *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval*

Models, arXiv:2104.08663, 2021. Available at: https://arxiv.org/abs/2104.08663.